AI 创新应用挑战赛 · 参赛材料

心屿（Isle）

技术文档

一座会记住你的心灵小岛

面向大学生的 AI 情绪支持应用



沧州交通学院 · 计算机与信息技术学院 · 软件工程 2410 · 吴思彤

v2.3.0 · 2026–11–06

目录

一、项目定位与边界

二、意图控制策略（Intent Control）

三、系统架构

四、端侧记忆引擎

五、多智能体安全路由

六、本地匿名账户

七、关键数据流

八、测试与验证

九、可复现性

十、已知限制与后续规划

附录 A · 关键文件

附录 B · 复现命令

一、项目定位与边界

1.1 产品定位

心屿是一款原生 Android **面向大学生的 AI 情绪支持应用**。

它用三种固定角色（闺蜜、老师、家人）和八个预设场景，帮助用户：

1. 把情绪说出来；
2. 把事实、感受和需求分开；
3. 找到一个今天能完成的小动作；
4. 在需要时被引导到真人支持。

1.2 明确不做的事

不做	原因
不提供恋人角色	避免恋爱/暧昧角色扮演与情感依赖风险
不做心理诊断 / 治疗	AI 不具备专业资质，不能替代心理咨询或医疗
不收集年龄 / 身份证 / 学籍	降低隐私与合规风险
不让模型决定危机路由	高危场景必须由确定性规则先行拦截

1.3 核心差异化

- **端侧语义记忆**：向量化、存储、召回都在手机本地；
- **安全路由**：高危/危机暂停角色扮演，不让模型自由发挥；
- **匿名本地账户**：不需要手机号、邮箱、实名；
- **四栏陪伴结构**：首页 / 场景 / 记忆 / 我的，降低使用门槛。

二、意图控制策略（Intent Control）

2.1 先定契约，再要代码

每次让 AI 生成代码前，先明确：

- 分层边界：UI / Domain / Data / 基础设施；
- 依赖约束：手写 `AppContainer`，不引入 DI 框架；

- 状态约束：ViewModel 用 `StateFlow` 单向输出；
- 失败降级：端侧模型、网络、账户都必须有失败路径；
- 验收标准：编译、单测、真机行为。

对应到运行时，提示词也采用“契约式”结构：

```
【总则（最高优先级）】
→ 不做诊断、不承诺疗效、不鼓励依赖、危机时安全优先

【角色卡（不可变）】
→ 名称 / 关系 / 性格 / 背景 / 说话风格

【当前模式】
→ 自由聊 / 倾听 / 分析 / 呼吸 / 场景 / 安全

【情绪信号】
→ 情绪标签 + 强度 + 风险等级

【记忆与用户资料】
→ 长期记忆 Top-6 + 全局资料

【表达约束】
→ 第一人称、口语化、不用 Markdown、长度适中
```

2.2 让编译器与测试当裁判

- 构建：`assembleDebug` / `compileDebugKotlin --rerun-tasks`；
- JVM 单测：安全路由、输出校验、密码、场景、URL、SSE、DTO；
- 仪器化测试：记忆隔离、端侧向量；
- 真机：注册、锁定、场景、危机拦截、记忆折叠、四栏导航。

当前测试结果：

```
JVM: 60 tests, 0 failures
Instrumented: 2 tests, 0 failures
```

2.3 报错即上下文

开发过程中坚持“报错原文 → 根因 → 修复 → 验证”的闭环，而不是只改表象。例如：

- Kotlin 元数据版本不兼容 → 反查依赖树，锁定 Coil 3.4.0；
- AGP 9 新 DSL 与独立 Kotlin 插件冲突 → `android.newDsl=false`；
- 消息同毫秒排序错位 → `ORDER BY timestamp DESC, id DESC`；
- 流式 `[DONE]` 与 `onClosed` 双回调 → `AtomicBoolean` 单次结算；

2.4 抽象先行，为失败预留降级路径

能力	接口	失败降级
端侧向量	`EmbeddingProvider`	关键词匹配
模型服务	`OpenAiClient`	明确错误提示 + 重试
安全评估	`SafetyEngine`	保守规则优先
本地账户	`LocalAuthRepository`	无法解密时回到注册页

三、系统架构

3.1 总体分层

```
UI（Compose）
  Screen（无状态）←→ ViewModel（StateFlow 单向数据流）
  四栏导航：首页 / 场景 / 记忆 / 我的

Domain
  chat      ConversationOrchestrator（多智能体编排）
  safety    SafetyEngine / ResponseValidator
  scenario  ScenarioCatalog
  memory    BertTokenizer / TextEmbedder / MemoryExtractor
  prompt    PromptBuilder

Data
  Room（characters / messages / memories）
  DataStore（AiSettings / UserProfile）
  OpenAiClient（OpenAI 兼容 / SSE / 多模态）
  LocalAuthRepository（本地账户 / PBKDF2 / 恢复码）

基础设施
  AppContainer（手写依赖容器）
  AvatarStorage（图片分级压缩）
  ONNX Runtime（端侧推理）
```

3.2 关键设计决策

决策	理由
单模块 + 包分层	项目规模可控，降低 AI 生成跨层依赖的概率
手写 `AppContainer`	结构透明，避免 DI 框架带来的隐式依赖

决策	理由
Room 三表	角色 / 消息 / 记忆职责清晰，外键级联删除
OpenAI 兼容协议	地址、模型、Key 可切换，不绑定单一厂商
Serverless 代理	真实 Key 不进 APK，由 Cloudflare Worker 持有并保护
本地匿名账户	不需要手机号/邮箱，符合情绪支持场景的隐私诉求
四栏导航	把“场景”和“记忆”从聊天深处提出来，降低发现成本

3.3 数据模型

characters

字段	类型	说明
id	Long	主键
name	String	角色名
relationType	String	bestie / teacher / family / friend / custom
personality	String	性格
bio	String	背景故事
speakingStyle	String	说话风格
greeting	String	开场白
avatarPath	String?	头像本地路径
isDefault	Boolean	是否内置
createdAt / updatedAt	Long	时间戳

messages

字段	类型	说明
id	Long	主键
characterId	Long	所属角色，外键级联
role	String	user / assistant
content	String	文本

字段	类型	说明
imagePath	String?	图片本地路径
timestamp	Long	时间戳

索引：`(characterId)`、`(characterId, timestamp)`。

memories

字段	类型	说明
id	Long	主键
characterId	Long	所属角色，外键级联
content	String	记忆内容
category	String	basic / preference / event / relationship / promise
importance	Float	0–1
embedding	ByteArray?	512 维 float 序列化
createdAt / lastUsedAt	Long	时间戳

数据库包含 `(characterId, timestamp)` 复合索引，覆盖更新时通过 Migration 保留已有聊天记录。

3.4 四栏导航

路由	内容
`characters`	首页：角色列表 + 快速场景
`scenarios`	完整 8 个场景
`memory_overview`	跨角色记忆总览
`me`	我的资料与账户、安全资源、关于心屿、设置
`chat/{characterId}`	聊天详情（隐藏底部栏）
`memory/{characterId}`	单角色记忆管理（隐藏底部栏）
`character_edit/{characterId}`	角色编辑（隐藏底部栏）

底部栏只在四个 Tab 路由显示；进入聊天、编辑、账户等二级页面自动隐藏，避免遮挡输入框。

3.5 启动流程

```
系统启动图标
  → 品牌封面（约 1.4 秒）
  → 本地账户：欢迎 / 登录 / 锁屏
  → 主界面（四栏导航）
```

应用锁：后台超过 60 秒回到前台需要重新输入密码；连续 5 次失败后锁定并递增等待。

3.6 模型服务接入与密钥保护

App 只依赖 OpenAI 兼容协议，接入方式有两种：

模式	适用场景	Key 位置
代理模式	公开分发、比赛演示	Cloudflare Worker Secret
BYOK	用户自带 Key、私有部署	用户本机 DataStore

公开分发包采用代理模式：

```
Android App (AI_BASE_URL + CLIENT_TOKEN)
  → Cloudflare Worker
      1. 校验 Authorization: Bearer <CLIENT_TOKEN>
      2. 校验模型白名单
      3. 限制 max_tokens
      4. 透传 SSE
  → 千问 MaaS (UPSTREAM_API_KEY 只在 Worker Secret)
```

关键约束：

- `UPSTREAM_API_KEY` 只通过 `wrangler secret put` 写入，不进入源码、`wrangler.toml` 或 APK；
- APK 只含代理地址与可撤销的 `CLIENT_TOKEN`；
- 代理不提供任意 URL 转发，只暴露 `/v1/models` 与 `/v1/chat/completions`；
- 请求体上限 10MB（兼容图片 base64），`max_tokens` 默认且最大 2048；
- 模型白名单当前为 `qwen3.5-omni-plus`、`deepseek-chat`、`deepseek-flash`；
- 当前不启用按 IP 限流，优先保证图片与长回复速度；
- 换上游 Key 只需更新 Secret，立即生效，不需要重新打包；
- 换 `CLIENT_TOKEN` 或代理地址才需要更新 `local.properties` 并重新构建。

安全边界：`CLIENT_TOKEN` 必须在客户端使用，因此仍可能被反编译提取。它能调用代理，但拿不到上游真实 Key，且受白名单与 `max_tokens` 约束。发现滥用时应轮换 `CLIENT_TOKEN` 并重新分发 APK。

四、端侧记忆引擎

4.1 模型与推理

环节	实现
模型	`bge-small-zh-v1.5` int8，23.9MB
维度	512
分词	WordPiece + CJK 逐字
池化	CLS + L2 归一化
推理	ONNX Runtime Android 1.31.0
存储	Room BLOB

4.2 写入与去重

```
LLM 抽取记忆 JSON
  → 内容精确去重
  → 端侧 embedding
  → 语义近似去重（余弦 ≥ 0.92）
  → 写入 Room
```

4.3 召回评分

```
score = semantic × 0.8 + lexical × 0.2 + importance × 0.1
```

- 候选集：最近 200 条；
- 取 Top-6 注入提示词；
- 向量不可用时，`semantic = 0`，自动退化为关键词 + 重要度。

4.4 角色隔离与折叠管理

- 每条记忆带 `characterId`；
- 查询、删除、清空都按角色过滤；
- 记忆 Tab 按角色折叠，默认收起；
- 点击角色展开只读列表；
- 「管理」按钮进入单角色记忆管理，可搜索、编辑、删除、清空；

- 展开状态保存在 ViewModel，返回详情/切换 Tab 不丢失。

4.5 安全约束

- 高危 / 危机对话不调用记忆抽取；
- 危机内容不进入长期记忆；
- 记忆措辞使用角色第一人称，避免“用户……”式冷记录。

4.6 情绪手账生成

```
最近对话（最多 20 条）
  → JournalExtractor：LLM 提炼金句 / 标题 / 情绪
  → 失败或无模型：回退到最后一条助手回复
  → JournalImageGenerator：1080×1440 Canvas 绘制
  → 品牌渐变背景 + 金句 + 情绪标签 + 角色落款
  → 右下角 ZXing 二维码
  → JournalSaver：写入 MediaStore Pictures/心屿
```

安全约束：

- 最近对话包含高危/危机内容时，不生成手账；
- 金句不超过 60 字；
- 二维码地址来自 `BuildConfig.SHARE_QR_URL`，默认指向项目仓库；
- 图片只写入本地相册，不自动上传。

五、多智能体安全路由

5.1 为什么需要路由

情绪支持应用最大的风险不是“回复不好”，而是：

- 高危表达被角色扮演掩盖；
- 模型给出危险建议；
- 危机时仍然说“我会一直陪着你”，却没有把用户交给真人。

因此，安全判断不能完全交给 LLM。

5.2 架构

```
用户输入
  → ① SafetyEngine：情绪 + 风险检测
  → ② ConversationOrchestrator：确定性路由
        ├─ CRISIS：固定危机话术，退出角色，不调用模型
        ├─ HIGH：安全确认 + 真人资源，不调用模型
        └─ NORMAL：进入角色/场景/倾听等模式
  → ③ PromptBuilder：角色卡 + 场景 + 情绪 + 记忆
  → ④ OpenAiClient：流式回复
  → ⑤ ResponseValidator：输出校验
```

这套结构与“多智能体”对应：

角色	职责
情绪/风险检测 Agent	本地规则优先，输出情绪与 L0–L3
安全策略路由 Agent	确定性状态机，决定正常/安全/危机
人设生成 Agent	按角色卡与模式生成回复
输出校验 Agent	拦截 AI 身份暴露、危险方法、倾听模式给建议
记忆管理 Agent	敏感过滤、写入/不写入

5.3 风险分级

等级	表现	处理
L0	日常情绪	正常角色回复
L1	焦虑、低落、压力	共情 + 倾听/场景模式
L2	无望感、强烈痛苦、自伤念头	安全确认 + 真人资源，不调用模型
L3	明确自伤/自杀表达	固定危机话术，退出角色，不调用模型

5.4 危机话术

固定包含：

- 12356 全国心理援助热线（以当地开通情况为准）；
- 120 / 110；
- 学校心理中心 / 辅导员；
- 身边可信任的人；
- 明确说明 AI 不能替代真人帮助。

危机内容不写入长期记忆；紧急情况不提供任何自伤方法、剂量、地点等细节。

5.5 输出校验

`ResponseValidator` 检查：

- 是否暴露 AI 身份 / 语言模型 / 系统提示词；
- 是否出现危险方法细节；
- 倾听模式是否给了建议；
- 是否使用 Markdown 或过长。

严重问题使用安全兜底文案，不直接落库。

5.6 角色一致性

- 角色卡不可变；
- 日常 / 倾听 / 分析 / 呼吸 / 场景 / 安全各有独立提示词模板；
- 危机模式“退出角色”是有意的安全切换，不是角色崩坏；
- 聊天页可查看结构化 Agent Trace，不展示模型内部推理。

六、本地匿名账户

6.1 设计目标

- 不需要手机号、邮箱、实名；
- 账户只用于本机身份识别与数据隔离；
- 密码、恢复码、账户数据不离开设备；
- 删除账户能真正清空本地数据。

6.2 密码与恢复码

项目	实现
密码哈希	PBKDF2WithHmacSHA256，随机 16 字节盐，210,000 次迭代
校验	常量时间比较
恢复码	128 位随机，分组显示，只存哈希，只显示一次
重新生成	验证当前密码后生成新码，旧码立即失效

项目	实现
失败锁定	5 次后 30 秒起，逐步递增
应用锁	后台超过 60 秒需要重新解锁

6.3 数据保护

- 账户 JSON 使用 Android Keystore 的 AES/GCM 密钥加密；
- 加密后的数据存入私有 SharedPreferences；
- 不保存明文密码或明文恢复码；
- `allowBackup=false`，不参与系统云备份。

6.4 删除账户

删除本地账户时清除：

- 账户与恢复码哈希；
- 角色、聊天、记忆；
- 头像与聊天图片；
- 模型服务配置；
- 用户资料。

6.5 能力边界

本地账户**不做**：

- 邮箱/手机号归属核验；
- 学籍/学生身份核验；
- 实名认证；
- 跨设备账号找回。

这些能力需要服务端与合规资质，超出当前本地应用范围。

七、关键数据流

7.1 一次对话

```
1. 校验账户 / 设置
2. 安全检查（SafetyEngine）
3. 高危/危机 → 固定话术，结束
4. 正常 → 落库用户消息
5. 召回 Top-6 记忆
6. 读取用户资料
7. PromptBuilder 组装
8. OpenAiClient SSE 流式返回
9. ResponseValidator 校验
10. 落库助手消息
11. 异步抽取记忆（安全模式跳过）
```

7.2 场景对话

```
首页/场景页 → 选场景 → 选角色
  → ChatViewModel 插入场景开场白
  → 模式 = SCENARIO
  → 提示词带“当前建议步骤”
  → 每轮只推进一个小步骤
```

7.3 记忆管理

```
记忆 Tab
  → 按角色折叠
  → 展开只读查看
  → 点「管理」
  → 单角色记忆管理
      → 搜索 / 编辑 / 删除 / 清空
```

八、测试与验证

8.1 自动化测试

类型	覆盖	结果
JVM	安全路由、输出校验、密码、恢复码、场景、URL、SSE、DTO、配置回退	**60 tests, 0 failures**

类型	覆盖	结果
Instrumented	记忆隔离、端侧向量	2 tests, 0 failures（真机）

关键测试：

- `SafetyEngineTest`：危机、高危、日常、否定表达、无空格英文；
- `ConversationOrchestratorTest`：危机优先级、安全模式退出、场景模式；
- `ResponseValidatorTest`：AI 身份暴露、危险方法、倾听模式建议；
- `PasswordHasherTest`：正确/错误密码、不同盐值、恢复码格式；
- `CharacterVisibilityTest`：恋人角色下线；
- `ScenarioCatalogTest`：场景完整性与唯一 ID。

8.2 真机验证

设备：CPH2747，Android 17，1272×2772。

已验证：

- 冷启动封面 → 登录/锁屏；
- 本地账户注册、错误密码锁定、正确密码解锁；
- 应用锁、手动锁定；
- 四栏导航切换；
- 场景选择角色 → AI 主动开场；
- 树洞模式提示与输入框占位；
- 流式回复；
- `iwanttodie` → 危机弹窗、角色扮演暂停、12356/120/110；
- 危机后 `iamsafenow` → 恢复自由聊；
- 记忆按角色折叠、展开、搜索、管理；
- 默认代理配置下不填写 Key，发送消息可正常流式回复；
- Worker `/v1/models` 200、无令牌 401、非白名单模型 400；
- APK 扫描不含 `sk-ws-` 上游 Key，只含代理地址与令牌；
- 新图标、底部导航、返回状态保持。

九、可复现性

9.1 环境

- JDK 17；
- Android SDK Platform 37 / Build-Tools 36；
- Gradle Wrapper 9.6.0；
- `local.properties` 配置 SDK 与模型服务（代理地址 + 代理令牌，或 BYOK 的真实 Key）；
- ONNX 模型随包分发。

9.2 构建与测试

```
./gradlew :app:testDebugUnitTest
./gradlew :app:assembleDebug
adb install -r app/build/outputs/apk/debug/app-arm64-v8a-debug.apk
```

9.3 测试数据

- 首次进入写入 3 个角色、6 条记忆、7 条示例对话；
- 设置页可重置演示数据。

十、已知限制与后续规划

限制	现状	后续
语音	未实现	TTS 朗读 + 语音输入
生物识别	未实现	BiometricPrompt
云同步	未实现	可选端到端加密同步
本地账户	无跨设备找回	可选云账号 / 加密导出
风险规则	本地规则可能有误判	扩充测试集 + 专业审核
内容审核	未接第三方	上架前接入审核与 AI 标识
发布工程	debug 签名	release keystore + R8 + versionCode
代理可用性	依赖 Cloudflare Worker	成本告警、备用域名、异常调用封禁

限制	现状	后续
客户端令牌	可被反编译提取	设备注册、短期 JWT、Play Integrity
数据备份	`allowBackup=false`	本地加密导出/导入

附录 A · 关键文件

模块	路径
安全评估	`domain/safety/SafetyEngine.kt`
多智能体编排	`domain/chat/ConversationOrchestrator.kt`
输出校验	`domain/safety/ResponseValidator.kt`
场景库	`domain/scenario/Scenario.kt`
提示词	`domain/prompt/PromptBuilder.kt`
聊天	`ui/screen/chat/ChatViewModel.kt`
记忆	`data/repository/MemoryRepository.kt`
本地账户	`data/auth/LocalAuthRepository.kt`
导航	`ui/navigation/IsleNavHost.kt`
启动封面	`ui/screen/splash/SplashScreen.kt`
图标	`res/drawable/ic_launcher_*.xml`
代理 Worker	`server/cloudflare-worker/worker.js`
Worker 配置	`server/cloudflare-worker/wrangler.toml`
代理部署说明	`server/cloudflare-worker/README.md`

附录 B · 复现命令

```
# 单测
./gradlew :app:testDebugUnitTest --offline

# 打包
./gradlew :app:assembleDebug --offline

# 安装
adb install -r app/build/outputs/apk/debug/app-arm64-v8a-debug.apk

# 真实接口 E2E（需 local.properties）
./gradlew :app:testDebugUnitTest --tests '*OpenAiClientE2eTest*' -Disle.e2e=1

# 代理部署 / 更新 Secret（在 server/cloudflare-worker 下）
npx wrangler deploy
npx wrangler secret put UPSTREAM_API_KEY
npx wrangler secret put CLIENT_TOKEN
```